Guidelines for Final Year Project Report

A Comprehensive Guide for Students

COMP1682 - Final Year Project

Report Structure Overview

Your final year project report consists of **9 major chapters**:

Chapter	Title	Focus
1	Introduction	Project overview, problem, aim, objectives
2	Background & Literature Review	Research context and gap analysis
3	Requirements Analysis	User needs and system requirements
4	Methodology & Planning	How you'll build and manage the project
5	System Design & Architecture	Technical blueprints and diagrams
6	Implementation	Code, database, deployment evidence
7	Testing	Quality assurance and test results
8	LSEPI	Legal, Social, Ethical, Professional Issues
9	Conclusion & Reflection	Achievements and learnings

Chapter 1: Introduction

Purpose

Set the stage for your entire project. This section should be **concise**, **engaging**, and clearly articulate the project's purpose.

Key Subsections

INTRODUCTION CHAPTER			
1.1 General Overview	1.2 Problem Statement	1.3 Project Aim	1.4 Specific Objectives
1.5 Scope & Limitations		1.6 Report Structure	

1.1 General Overview

What to Include

- Introduce your project's **domain** (e.g., FinTech, e-commerce, mobile health)
- Explain **relevance** in the current landscape
- Hook the reader with compelling context

Example (eCommerce Project)

> "The global e-commerce market has experienced unprecedented growth, reaching $5.8 trillion in 2023. Small and medium businesses increasingly seek affordable digital solutions to compete in this expanding marketplace. This project focuses on developing an accessible e-commerce platform tailored for local businesses in emerging markets."

1.2 Problem Statement

The Core of Your Introduction

Your problem statement must clearly answer:

Question	What to Address
What?	The specific problem, gap, or challenge
Who?	Who is affected by this problem?
Why?	Why does this problem need solving?
Evidence?	Statistics or references supporting the problem

Do's ✓

- Be **precise** and **specific**
- Support with data/references
- Focus on ONE main problem

Don'ts ✗

- Avoid vague statements
- Don't list multiple unrelated problems

1.2 Problem Statement: Example

❌ Weak Statement

> "Small businesses have problems selling online."

✅ Strong Statement

> "Despite the growing e-commerce opportunity, 67% of small local businesses in Vietnam lack the technical expertise and financial resources to establish an online presence (VECOM, 2024). Current platforms like Shopify and WooCommerce require significant setup costs and technical knowledge, creating a barrier for micro-enterprises with limited budgets. This digital divide prevents local businesses from accessing the broader market, resulting in lost revenue and reduced competitiveness against larger, digitally-enabled competitors."

1.3 Project Aim

One Comprehensive Sentence

Your aim should cover:

- **What** you are building
- **Who** it is for
- **Why** it matters (the benefit)

Formula: "To [action verb] a [type of system] that [does what] for [target users] in order to [achieve benefit]."

Example

> "To design, develop, and deploy a user-friendly e-commerce web application that enables small local businesses to create online stores, manage product inventory, and process customer orders efficiently, thereby reducing the technical and financial barriers to digital commerce adoption and helping local entrepreneurs expand their market reach."

1.4 Specific Objectives

Format: "To + Verb"

Each objective must be **SMART**:

- **S**pecific - Clear action
- **M**easurable - Quantifiable outcome
- **A**chievable - Realistic
- **R**elevant - Connected to aim
- **T**ime-bound - Has deadline

⚠️ **Important:** In the final report (unlike proposal), you CAN mention specific technologies since you've already made your choices!

Common Verbs: To design, To develop, To implement, To integrate,

1.4 Objectives: Example

#	Objective
1	To design and develop a responsive web application using React.js for the frontend and Node.js for the backend
2	To implement user authentication and authorization using JWT for secure access control
3	To create a RESTful API for managing products, orders, and customer data
4	To integrate a payment gateway (Stripe/PayPal) for secure online transactions
5	To deploy the application on cloud infrastructure and conduct user acceptance testing with at least 10 participants

Remember: Each objective should be traceable - you'll evaluate these in Chapter 9!

1.5 Scope and Limitations

Define Clear Boundaries

In-Scope (Will Deliver)	Out-of-Scope (Won't Cover)
User registration & login	Mobile native application
Product catalog management	Multi-vendor marketplace
Shopping cart & checkout	Advanced AI recommendations
Basic order tracking	Real-time chat support
Admin dashboard	Multi-currency support

Limitations

Be honest about constraints:

- Time constraints (12-week development period)
- Technical constraints (single developer, limited hosting budget)
- Data constraints (testing with synthetic data)

Tip: This prevents "scope creep" and sets realistic expectations!

1.6 Report Structure

Briefly Outline the Report

Provide a roadmap for readers:

> "The remainder of this report is structured as follows:
>
> **Chapter 2** presents the background research and literature review...
> **Chapter 3** details the requirements analysis and user research...
> **Chapter 4** describes the methodology and project planning...
> **Chapter 5** covers the system design and architecture...
> **Chapter 6** documents the implementation process...
> **Chapter 7** presents the testing strategy and results...
> **Chapter 8** discusses legal, social, ethical, and professional issues...
> **Chapter 9** concludes with achievements, reflection, and future work..."

Keep it brief: 1 paragraph or a simple list is sufficient.

Chapter 2: Background & Literature Review

Purpose

Critically analyze existing work to establish context and **justify the need** for your project.

Structure Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                BACKGROUND & LITERATURE REVIEW                        │
├──────────────────────────────────────────────────────────────────────┤
│  2.1 Introduction (Purpose, Scope, Research Strategy)                │
│  2.2 Problem Domain Analysis (Context, Trends, Concepts)             │
│  2.3 Critical Review of Existing Solutions (Competitors)             │
│  2.4 Critical Review of Technologies & Methodologies                 │
│  2.5 Summary and Identification of Research Gap                      │
└──────────────────────────────────────────────────────────────────────┘
```

2.1 Introduction to Literature Review

Three Key Components

Component	What to Include	Example
Purpose Statement	Clear statement of intent	"This chapter critically reviews existing literature to establish context and justify the proposed project."
Scope of Research	Boundaries of your review	"The review focuses on e-commerce platforms for SMEs released in the last 5 years..."
Research Strategy	How you found sources	"Sources were gathered from ACM Digital Library, IEEE Xplore, and industry reports..."

Credible Sources

✓ Academic databases (ACM, IEEE, Google Scholar)

✓ Industry reports (Gartner, Statista, McKinsey)

✓ Official documentation

✗ Wikipedia, random blogs (as primary sources)

2.2 Problem Domain Analysis

The "Big Picture" Context

What to Cover

Historical Context & Current Trends

- How has this domain evolved?
- What technological/social trends are driving its importance?

Key Concepts & Terminology

- Define core theories, models, technical terms
- Don't assume the reader knows everything

Significance & Impact

- Use statistics from credible sources
- Who is affected? What are the consequences?

Example

> "According to Statista (2024), global e-commerce sales are projected to reach $8.1 trillion by 2026, with mobile commerce accounting for 72.9% of total e-commerce sales..."

2.3 Critical Review of Existing Solutions

Analyze the Competition

Goal: Identify weaknesses to create space for YOUR project.

Steps

1. **Identify 2-4 competitors** (commercial products or academic research)
2. **Create a Comparison Matrix** (systematic analysis)
3. **Write a narrative analysis** (interpret, don't just describe)

Comparison Matrix Template

Feature	Solution A	Solution B	Solution C	Your Project
Target Audience				
Core Technology				
Key Features				
Strengths				
Weaknesses				How you overcome

2.3 Comparison Matrix: Example

Feature	Shopify	WooCommerce	Haravan	Your Project
Target Audience	Global SMEs	WordPress users	Vietnam SMEs	Local micro-businesses
Pricing	$29-299/month	Free + hosting	$200K+ VND/month	Free tier available
Tech Required	Low	Medium-High	Low	Very Low
Local Payment	Limited	Plugin-based	Yes	Yes (VNPay, Momo)
Vietnamese UI	Partial	Plugin-based	Yes	Yes (Native)
Strengths	Feature-rich	Flexible	Local support	Simple, affordable
Weaknesses	Expensive, not localized	Needs technical skills	Still costly for micro	Limited advanced features

2.3 Written Analysis: Example

Synthesize Your Findings

Don't just repeat the table - **interpret** it:

> "As the comparison matrix illustrates, a significant gap exists in the Vietnamese e-commerce platform market. While Shopify offers comprehensive features, its pricing model ($29-299/month) is prohibitive for micro-businesses with monthly revenues under 10 million VND. WooCommerce provides a free alternative but requires technical expertise that most local shop owners lack.
>
> Haravan addresses the localization needs but maintains pricing that still excludes the smallest businesses. None of the existing solutions adequately serve the micro-enterprise segment that represents 76% of Vietnamese businesses (GSO, 2024).

2.4 Critical Review of Technologies

Justify Your Technical Choices

Compare options and explain **why** you chose each technology.

Areas to Cover

Category	Compare	Criteria
Frontend	React vs Vue vs Angular	Performance, ecosystem, learning curve
Backend	Node.js vs Python vs PHP	Speed, scalability, database compatibility
Database	SQL vs NoSQL	Data structure, ACID compliance, flexibility
Methodology	Agile vs Waterfall	Project requirements, iteration needs

Key: Every choice needs a **justified reason** with references!

2.4 Technology Comparison: Example

Frontend Framework Comparison

Criteria	React	Vue.js	Angular
Learning Curve	Medium	Low	High
Performance	High	High	Medium
Ecosystem	Very Large	Growing	Large
Community Support	Excellent	Good	Good
Job Market	Highest	Growing	High

Justification:

> "React was selected for its extensive ecosystem, including state management tools like Redux, and its component-based architecture which enables reusable UI components. According to the Stack Overflow Developer Survey (2024), React remains the most commonly used web framework at 40.6% adoption."

2.4 Development Methodology Comparison

2.4 Methodology Justification

Why Agile/Scrum?

Factor	Waterfall	Agile/Scrum
Requirements	Fixed upfront	Can evolve
Flexibility	Low	High
User Feedback	At the end	Every sprint
Risk Discovery	Late	Early
Documentation	Heavy	Light

Justification:

> "Agile Scrum was chosen because this project has evolving requirements based on user feedback. The iterative approach allows for continuous improvement through regular sprint reviews, which is essential for developing a user-centered e-commerce platform. According to the Standish Group CHAOS Report (2020), Agile projects are 1.5x more successful than Waterfall projects."

2.5 Summary and Research Gap

The Most Critical Part of Chapter 2

Three Steps

1. Synthesize Your Findings

- Bring together main points from solutions, technology, and market analysis

2. State the Gap Explicitly

- Clear, confident statement of what's missing

3. Position Your Project

- How your project fills that gap

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│   Existing Solutions  ──→  Weaknesses/Gaps Identified        │
│                                   │                          │
│                                   ▼                          │
│   Your Project        ──→  Addresses These Gaps              │
│                                   │                          │
│                                   ▼                          │
│   Unique Value        ──→  Your Contribution                 │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

2.5 Research Gap: Example

Stating the Gap

"In summary, the literature review has identified a clear gap in the Vietnamese e-commerce market: **there is no affordable, easy-to-use e-commerce platform specifically designed for micro-businesses** with limited technical expertise and minimal startup capital.

Existing solutions are either too expensive (Shopify, Haravan), too technically complex (WooCommerce), or lack essential local features (payment integration, Vietnamese language support).

Therefore, this project aims to address this gap by developing a simplified e-commerce web application that offers:

- Zero-cost entry tier for micro-businesses
- No-code store setup wizard
- Native Vietnamese language interface
- Integration with local payment methods (VNPay, MoMo, ZaloPay)"

Chapter 3: Requirements Analysis

Purpose

Understand **what** to build by researching user needs and defining clear requirements.

Structure

```
┌──────────────────────────────────────────────────────────────────────┐
│                    REQUIREMENTS ANALYSIS                             │
├──────────────────────────────────────────────────────────────────────┤
│  3.1 Research Methodology                                            │
│      └─→ How did you gather data?                                    │
│  3.2 Market Analysis and User Needs                                  │
│      └─→ What did you discover? User personas                        │
│  3.3 Functional Requirements (MoSCoW)                                │
│      └─→ User stories with prioritization                            │
│  3.4 Non-functional Requirements                                     │
│      └─→ Performance, security, usability                            │
└──────────────────────────────────────────────────────────────────────┘
```

3.1 Research Methodology

Document Your Data Collection

Aspect	What to Include
Method	Survey, interviews, observation, etc.
Tool	Google Forms, Typeform, etc.
Sample Size	How many responses/participants?
Demographics	Age, occupation, location, etc.
Selection	How did you choose participants?
Duration	When was data collected?

Example

> "A mixed-methods approach was used. An online survey via Google Forms collected 47 responses over 2 weeks from small business owners in Da Nang. Additionally, 5 semi-structured interviews were conducted with local shop owners to gain deeper insights into their pain points and requirements."

3.2 Market Analysis and User Needs

Present Your Findings

Key Elements

1. **Visualize data** with charts/graphs
2. **Interpret** what the data means
3. **Create User Personas** based on findings

Survey Results Example

> "The survey revealed that:
>
> - **78%** of respondents found existing platforms 'too expensive'
> - **65%** rated their technical skills as 'beginner' or 'none'
> - **82%** preferred Vietnamese language interface
> - **71%** wanted integration with local payment methods
>
> This quantitative data strongly supports our focus on affordability, simplicity, and localization."

3.2 User Persona Template

Give Your User a Face

```
┌──────────────────────────────────────────────────────────────┐
│ 👤 PERSONA: [Name]                                           │
│ ──────────────────────────────────────────────────────────── │
│ 📋 Bio:                                                      │
│    [Short paragraph about their life and background]         │
│                                                              │
│ 🎯 Goals:                                                    │
│    • [What they want to achieve with your product]           │
│    • [Another goal]                                          │
│                                                              │
│ 😤 Frustrations:                                             │
│    • [Pain points with current solutions]                    │
│    • [Another frustration]                                   │
│                                                              │
│ 💪 Motivations:                                              │
│    • [What drives them]                                      │
└──────────────────────────────────────────────────────────────┘
```

3.2 User Persona: Example

👤 Persona: Nguyen Thi Lan

Age: 35 | **Occupation:** Small bakery owner | **Location:** Da Nang

Bio: Lan runs a small home bakery business selling cakes for local events. She currently takes orders via Facebook Messenger and Zalo, manually tracking orders in a notebook. She wants to expand but struggles with the technical aspects of going online.

Goals:

- Sell products online without technical knowledge
- Manage orders efficiently without manual tracking
- Accept digital payments (MoMo, bank transfer)

Frustrations:

- Existing platforms are too expensive or complicated
- Loses track of orders during busy periods
- Misses potential customers who prefer online shopping

Motivations:

- Grow her business beyond local word-of-mouth
- Spend less time on administration, more on baking

3.3 Functional Requirements (MoSCoW)

User Stories with Prioritization

Format: "As a [user], I want [something], so that [why]"

MoSCoW Prioritization:

- **M**ust Have - Critical for launch
- **S**hould Have - Important but not critical
- **C**ould Have - Nice to have
- **W**on't Have - Out of scope (this time)

```
┌──────────────────────────────────────────────────────────────┐
│              PRIORITIZATION PYRAMID                          │
│                       /\                                     │
│                      /  \                                    │
│                     /MUST\     ← Do First                   │
│                    /──────\                                  │
│                   / SHOULD \   ← Do If Time                 │
│                  /──────────\                                │
│                 /   COULD    \ ← Nice to Have                │
│                /──────────────\                              │
│               /     WON'T      \ ← Future Version            │
│              /────────────────────\                          │
└──────────────────────────────────────────────────────────────┘
```

3.3 Functional Requirements: Table

ID	Description	Priority	Est.
USR_01	As a customer, I want to browse products by category, so that I can find items easily	Must	5 days
USR_02	As a customer, I want to add items to cart, so that I can purchase multiple items	Must	3 days
USR_03	As a customer, I want to checkout and pay online, so that I can complete my purchase	Must	8 days
USR_04	As a customer, I want to track my order status, so that I know when to expect delivery	Should	3 days
ADM_01	As an admin, I want to add/edit products, so that I can manage my catalog	Must	5 days
ADM_02	As an admin, I want to view order reports, so that I can track business performance	Should	3 days
ADM_03	As an admin, I want to send promotional emails, so that I can market to customers	Could	5 days

3.3 Requirements Summary Table

Summarize Your Priorities

Priority	Number of Features	Estimated Days
Must	4	21 days
Should	3	9 days
Could	2	8 days
Won't	2	-
TOTAL	11	38 days

Analysis:

> "The Must-have features require 21 days of development, which fits within the 8-week implementation phase. Should-have features will be added if time permits after core functionality is complete. Could-have features are planned for future iterations."

3.4 Non-functional Requirements

Quality Attributes

ID	Category	Requirement
NF_01	**Performance**	Page load time should be under 3 seconds on 3G connections
NF_02	**Performance**	System should handle 100 concurrent users
NF_03	**Security**	All passwords must be hashed using bcrypt
NF_04	**Security**	All API endpoints must use HTTPS
NF_05	**Usability**	Users should complete checkout in under 5 clicks
NF_06	**Usability**	Interface must be responsive (mobile-friendly)
NF_07	**Reliability**	System uptime should be 99%
NF_08	**Scalability**	Database should support up to 10,000 products

Tip: Make requirements **measurable** - avoid vague terms like "fast" or "secure"

Chapter 4: Methodology & Project Planning

Purpose

Explain **how** you will build and manage the project, and how you'll measure success.

Structure

```
┌──────────────────────────────────────────────────────────────┐
│                METHODOLOGY & PROJECT PLANNING                │
├──────────────────────────────────────────────────────────────┤
│  4.1 Development Methodology                                 │
│      └─→ Agile/Scrum: How you applied it                     │
│  4.2 Project Plan & Timeline                                 │
│      └─→ Gantt chart, milestones, sprints                    │
│  4.3 Feasibility Analysis                                    │
│      └─→ Technical, Economic, Operational                    │
│  4.4 Evaluation Plan & Success Metrics                       │
│      └─→ How will you measure success?                       │
└──────────────────────────────────────────────────────────────┘
```

4.1 Development Methodology

Describe Your Practical Application

Don't just define Agile - explain **how you used it**:

Agile Element	How You Applied It
Sprints	2-week sprints, 6 sprints total
Sprint Planning	Beginning of each sprint, defined user stories
Daily Standups	Self-check progress tracking via Trello
Sprint Review	Demo to supervisor every 2 weeks
Retrospective	Reflected on what went well/poorly
Product Backlog	Maintained in Trello/GitHub Projects

Example:

> "The project followed Agile Scrum methodology with 2-week sprints. Each sprint began with planning to select user stories from the backlog. Progress was tracked using a Kanban board in Trello. Sprint reviews with the supervisor provided regular feedback opportunities."

4.2 Project Plan & Timeline

Gantt Chart

```
PHASE                        |W1|W2|W3|W4|W5|W6|W7|W8|W9|W10|W11|W12|
-----------------------------|--|--|--|--|--|--|--|--|--|---|---|---|
Discovery & Research         |▓▓|▓▓|  |  |  |  |  |  |  |   |   |   |
Requirements Analysis        |  |▓▓|▓▓|  |  |  |  |  |  |   |   |   |
System Design                |  |  |▓▓|▓▓|  |  |  |  |  |   |   |   |
Sprint 1 (Auth + Setup)      |  |  |  |▓▓|▓▓|  |  |  |  |   |   |   |
Sprint 2 (Products)          |  |  |  |  |▓▓|▓▓|  |  |  |   |   |   |
Sprint 3 (Cart + Order)      |  |  |  |  |  |▓▓|▓▓|  |  |   |   |   |
Sprint 4 (Payment)           |  |  |  |  |  |  |▓▓|▓▓|  |   |   |   |
Testing & Bug Fixes          |  |  |  |  |  |  |  |▓▓|▓▓|   |   |   |
User Acceptance Testing      |  |  |  |  |  |  |  |  |▓▓|▓▓|   |   |
Deployment                   |  |  |  |  |  |  |  |  |  |▓▓|   |   |
Report Writing               |  |  |  |  |  |  |  |▓▓|▓▓|▓▓|▓▓|▓▓|
Buffer/Contingency           |  |  |  |  |  |  |  |  |  |  |▓▓|▓▓|
```

4.3 Feasibility Analysis

Assess Project Viability

```
                    FEASIBILITY TRIANGLE
                          /\
                         /  \
                        /TIME\
                       /  &   \
                      / SKILLS \
                     /──────────\
                    /\          /\
                   /  \        /  \
                  /TECH\      /ECO-\
                 /NICAL\    /NOMIC \
                /────────\ /────────\
```

Dimension	Assessment
Technical	✓ Familiar with React, Node.js; Have VS Code, GitHub access
Economic	✓ Free tier hosting (Vercel, Render); No paid APIs required
Operational	✓ Solution is maintainable; Clear documentation planned

4.4 Evaluation Plan & Success Metrics

Define Success BEFORE Building

Quantitative Metrics (Measurable)

Metric	Target	How to Measure
Task Completion Rate	> 80%	User Acceptance Testing
Page Load Time	< 3 seconds	Lighthouse audit
Test Coverage	> 70%	Jest coverage report
Bug Count	< 5 critical bugs	Issue tracking

Qualitative Metrics (Subjective)

Metric	Target	How to Measure
User Satisfaction	> 4/5 average	Post-test survey (SUS)
UI Intuitiveness	Positive feedback	User interviews

Tip: These metrics will be evaluated in Chapter 9!

Chapter 5: System Design & Architecture

Purpose

Provide the **blueprint** for your system. Use diagrams extensively!

Structure

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SYSTEM DESIGN & ARCHITECTURE                      │
├──────────────────────────────────────────────────────────────────────┤
│  5.1 Rich Picture                                                    │
│      └─→ Visual overview of the problem/solution                     │
│  5.2 System Architecture                                             │
│      └─→ C4 Model: Context + Container diagrams                      │
│  5.3 Technology Stack                                                │
│      └─→ Justified tech choices with references                      │
│  5.4 Detailed Design Diagrams                                        │
│      └─→ ERD, Use Case, Activity diagrams                            │
│  5.5 API Design                                                      │
│      └─→ Endpoint documentation                                      │
└──────────────────────────────────────────────────────────────────────┘
```

5.1 Rich Picture

Visual Problem/Solution Overview

A Rich Picture shows the **context** of your system:

- Stakeholders and their concerns
- External systems
- Data flows
- Problem areas

Tips:

- Use icons, stick figures, speech bubbles
- Show relationships and interactions
- Highlight pain points with clouds/symbols
- Keep it informal but informative

Tools: Draw.io, Miro, Lucidchart, or hand-drawn

> Include your Rich Picture diagram and briefly explain what it shows about your product and its context.

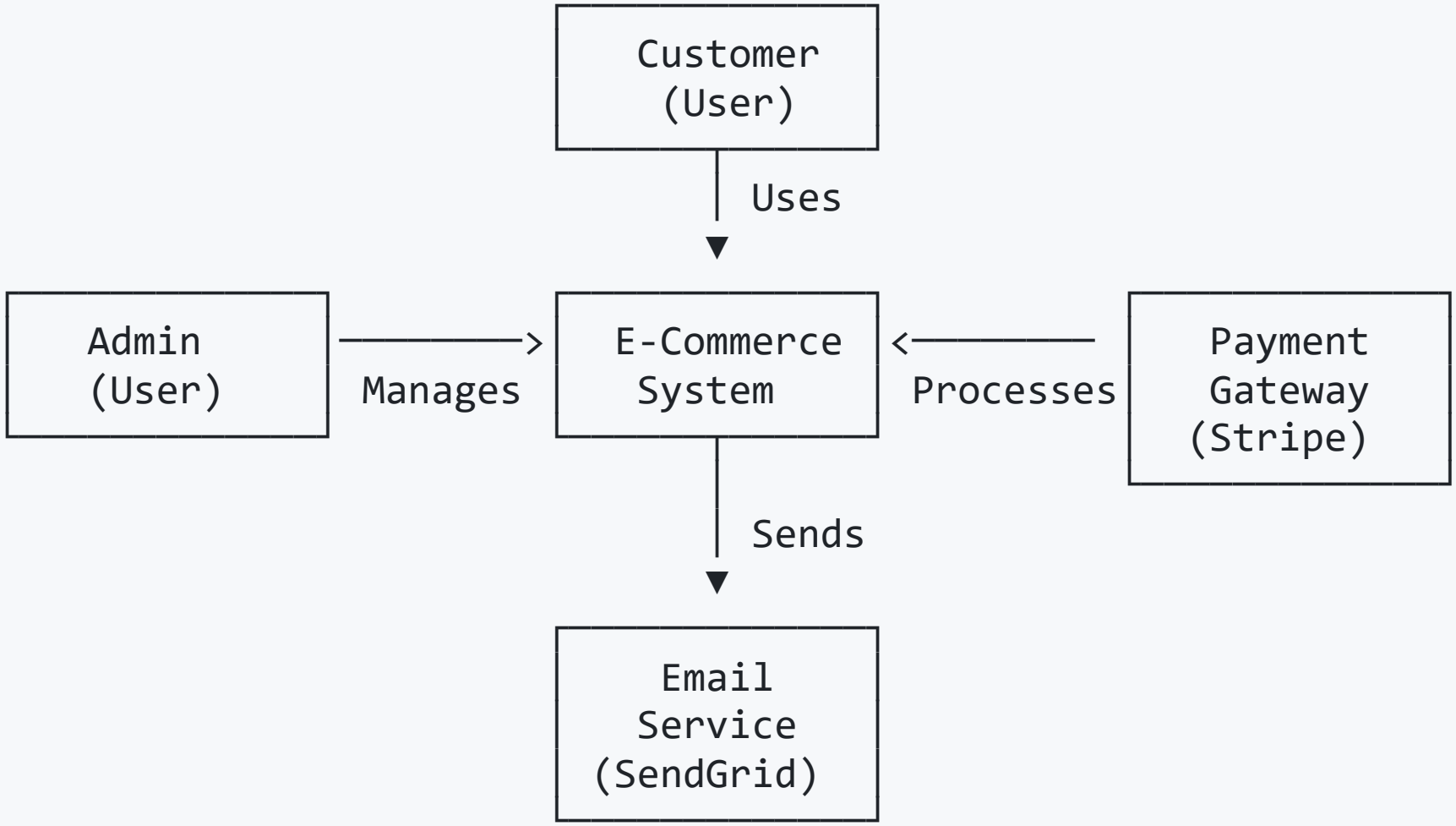
5.2 System Architecture

C4 Model Overview

The C4 Model provides different levels of abstraction:

```
┌─────────────────────────────────────────────────────────────────────┐
│  Level 1: SYSTEM CONTEXT
│  └─→ How your system fits in the world (users, externals)
│
│  Level 2: CONTAINER
│  └─→ Major building blocks (web app, API, database)
│
│  Level 3: COMPONENT
│  └─→ Components within containers (optional for FYP)
│
│  Level 4: CODE
│  └─→ Class/code level (optional for FYP)
└─────────────────────────────────────────────────────────────────────┘
```

5.2 System Context Diagram: Example



5.2 Container Diagram: Example

```
┌──────────────────────────────────────────────────────────────┐
│                        E-Commerce System                     │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│                                                              │
│   ┌──────────────────┐          API Calls                    │
│   │   Frontend       │   <──────────────────>   ┌──────────────────┐
│   │   (React)        │                          │   Backend        │
│   │                  │                          │  (Node.js +      │
│   │  - Customer      │                          │   Express)       │
│   │    pages         │                          │                  │
│   │  - Admin         │                          │  - REST API      │
│   │    dashboard     │                          │  - Auth JWT      │
│   └──────────────────┘                          │  - Business      │
│                                                 │    logic         │
│                                                 └──────────────────┘
│                                                          │
│                                                  Queries │
│                                                          ▼
│                                                 ┌──────────────────┐
│                                                 │   Database       │
│                                                 │  (MongoDB)       │
│                                                 └──────────────────┘
└──────────────────────────────────────────────────────────────┘
```

5.3 Technology Stack

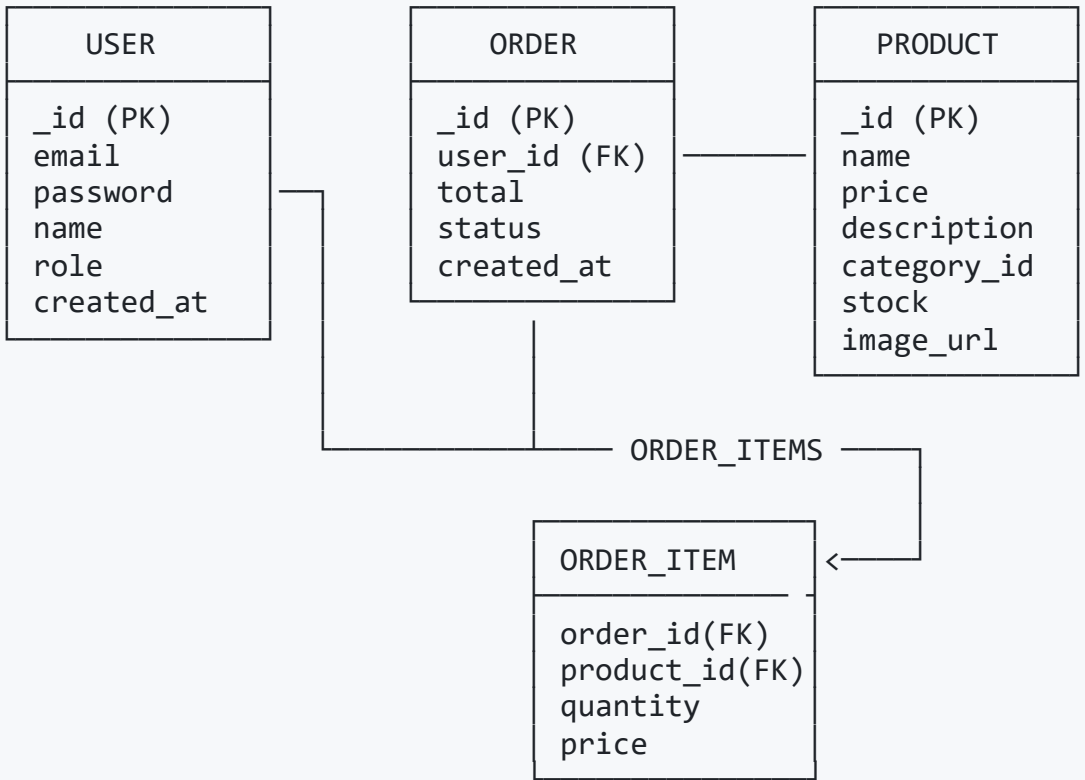
Present Your Full Stack with Justification

Category	Technology	Justification	Reference
Frontend	React 18	Component-based architecture, large ecosystem	React Docs
Styling	Tailwind CSS	Utility-first, rapid development	Tailwind Docs
Backend	Node.js + Express	JavaScript consistency, fast I/O	Node.js Docs
Database	MongoDB Atlas	Flexible schema, easy scaling	MongoDB Docs
Auth	JWT + bcrypt	Stateless auth, secure password hashing	OWASP
Payment	Stripe API	Industry standard, extensive documentation	Stripe Docs
Hosting	Vercel (FE), Render (BE)	Free tier, CI/CD integration	-
Version Control	Git + GitHub	Industry standard, collaboration	-
Testing	Jest + Postman	Unit testing + API testing	-

5.4 Detailed Design Diagrams

Entity Relationship Diagram (ERD)

Show the blueprint of your database:



5.4 Use Case Diagram

Show User Interactions

```
┌──────────────────────────────────────────────────────────────────────┐
│                        E-Commerce System                             │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │                                                              │    │
│  │    (Browse Products)──────────┐                              │    │
│  │                               │                              │    │
│  │    (Search Products)──────────┼──────────── Customer         │    │
│  │                               │                              │    │
│  │    (Add to Cart)──────────────┤                              │    │
│  │                               │                              │    │
│  │    (Checkout & Pay)───────────┘                              │    │
│  │                               ┌                              │    │
│  │    (Manage Products)──────────┐                              │    │
│  │                               │                              │    │
│  │    (View Orders)──────────────┼──────────────Admin           │    │
│  │                               │                              │    │
│  │    (Update Order Status)──────┘                              │    │
│  │                                                              │    │
│  └──────────────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────────┘
```

5.5 API Design

Document Key Endpoints

ID	URL	Method	Description	Auth
1	/api/auth/register	POST	Register new user	No
2	/api/auth/login	POST	User login, returns JWT	No
3	/api/products	GET	Get all products	No
4	/api/products/:id	GET	Get product by ID	No
5	/api/products	POST	Create new product	Admin
6	/api/cart	GET	Get user's cart	User
7	/api/cart	POST	Add item to cart	User

Tip: Include sample request/response bodies in your report!

Chapter 6: Implementation

Purpose

Document **how you built** your system with evidence.

Structure

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                IMPLEMENTATION                            │
├──────────────────────────────────────────────────────────────────────────┤
│  6.1 Development Environment                                             │
│      └─→ Tools, IDEs, libraries used                                     │
│  6.2 Backend & Frontend Implementation                                   │
│      └─→ Folder structure, code samples                                  │
│  6.3 Database Implementation                                             │
│      └─→ How you created the physical database                           │
│  6.4 Deployment                                                          │
│      └─→ Steps taken, live application screenshots                       │
│  6.5 Project Management                                                  │
│      └─→ Trello/GitHub screenshots, commit history                       │
└──────────────────────────────────────────────────────────────────────────┘
```

6.1 Development Environment

List Your Tools

Category	Tool	Purpose
IDE	VS Code	Code editing with extensions
Version Control	Git + GitHub	Source code management
Project Management	Trello	Task tracking (Kanban)
API Testing	Postman	Testing API endpoints
Browser DevTools	Chrome DevTools	Frontend debugging
Terminal	Windows Terminal	Command line operations
Design	Figma	UI/UX mockups
Database GUI	MongoDB Compass	Database management

Include: Key VS Code extensions, npm packages used

6.2 Backend & Frontend Implementation

Folder Structure

Show screenshots and explain the organizational logic:

Backend Structure:

```
server/
├── controllers/     # Request handlers
├── models/          # Database schemas
├── routes/          # API route definitions
├── middleware/      # Auth, validation middleware
├── utils/           # Helper functions
├── config/          # Configuration files
└── server.js        # Entry point
```

Frontend Structure:

```
client/
├── src/
│   ├── components/  # Reusable UI components
│   ├── pages/       # Page components
│   ├── hooks/       # Custom React hooks
│   ├── context/     # Global state (Context API)
│   ├── services/    # API call functions
│   └── utils/       # Helper functions
└── public/          # Static assets
```

6.2 Code Samples

Include Meaningful Examples

Choose complex or core functions. Add comments explaining the code:

```
// Example: Authentication middleware (server/middleware/auth.js)
const jwt = require('jsonwebtoken');

const authMiddleware = (req, res, next) => {
  // Extract token from Authorization header
  const token = req.header('Authorization')?.replace('Bearer ', '');

  if (!token) {
    return res.status(401).json({ error: 'Access denied' });
  }

  try {
    // Verify token and attach user to request
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (error) {
    res.status(400).json({ error: 'Invalid token' });
  }
};
```

6.3 Database Implementation

From ERD to Physical Database

Explain how you created the database:

> "The MongoDB database was created based on the ERD design. Mongoose ODM was used to define schemas with validation rules. The database is hosted on MongoDB Atlas (free tier) with the following collections:
>
> - users, products, categories, orders, order_items"

Include:

- Screenshot of MongoDB Compass/Atlas showing collections
- Sample document structure
- Indexes created for performance

```
// Example: Product Schema (server/models/Product.js)
const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: { type: Number, required: true, min: 0 },
  category: { type: mongoose.Schema.Types.ObjectId, ref: 'Category' },
  stock: { type: Number, default: 0 },
  // ... more fields
}, { timestamps: true });
```

6.4 Deployment

Document Your Deployment Process

Steps to include:

1. Environment setup (environment variables)
2. Build process
3. Deployment platform configuration
4. Domain/SSL setup (if applicable)

Example:

> "The frontend was deployed to Vercel with automatic CI/CD from GitHub.
> The backend was deployed to Render.com with the following steps:
>
> 1. Created a new Web Service on Render
> 2. Connected GitHub repository
> 3. Configured environment variables (MONGODB_URI, JWT_SECRET, etc.)
> 4. Deployed with automatic builds on push"

Include: Screenshots of the live, working application!

6.5 Project Management

Provide Evidence of Your Process

Screenshots to include:

- Trello/GitHub Projects board (showing task organization)
- Sprint/iteration boards
- GitHub commit history (showing consistent work)
- Pull requests (if applicable)

Good commit history example:

```
✓ feat: implement user registration API
✓ feat: add product listing page with pagination
✓ fix: resolve cart total calculation bug
✓ feat: integrate Stripe payment gateway
✓ test: add unit tests for auth middleware
✓ docs: update API documentation
```

Tip: Consistent, meaningful commits show professional development!

Chapter 7: Testing

Purpose

Demonstrate that your application is **robust and reliable**.

Structure

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                   TESTING                                │
├──────────────────────────────────────────────────────────────────────────┤
│  7.1 Testing Strategy                                                    │
│      └─→ Types of testing performed                                      │
│  7.2 Test Cases and Results                                              │
│      └─→ Detailed test case table with results                           │
└──────────────────────────────────────────────────────────────────────────┘
```

Testing Types

Type	Purpose	Example
Unit Testing	Test individual functions	Jest for JS functions
Integration Testing	Test components together	API endpoint testing
User Acceptance Testing	Test with real users	Task-based scenarios

7.1 Testing Strategy

Describe Your Approach

> "A multi-layered testing approach was employed:
>
> **Unit Testing:** Individual functions and components were tested using Jest. Focus was on authentication logic, cart calculations, and data validation functions. Target: 70% code coverage.
>
> **Integration Testing:** API endpoints were tested using Postman to ensure proper request handling, error responses, and data flow between frontend and backend.
>
> **User Acceptance Testing (UAT):** 10 participants matching our user persona profile were recruited to test the application. They completed predefined tasks while being observed. A post-test survey collected feedback on usability and satisfaction."> predefined tasks while being observed. A post-test survey collected feedback on usability and satisfaction."

7.2 Test Cases and Results

Present Your Test Cases

ID	Description	Steps	Expected	Actual	Pass/Fail
TC-01	Successful Login	1. Enter valid email 2. Enter valid password 3. Click Login	Redirect to dashboard, show welcome message	As expected	✓ Pass
TC-02	Login - Wrong Password	1. Enter valid email 2. Enter wrong password 3. Click Login	Show error "Invalid credentials"	As expected	✓ Pass
TC-03	Add to Cart	1. Browse products 2. Click "Add to Cart" 3. View cart	Product appears in cart with correct price	As expected	✓ Pass
TC-04	Checkout Empty Cart	1. Empty cart 2. Click Checkout	Show error "Cart is empty"	As expected	✓ Pass
TC-05	Complete Purchase	1. Add items 2. Checkout 3. Enter payment	Order created, confirmation shown	As expected	✓ Pass

7.2 UAT Results Summary

User Acceptance Testing Results

Metric	Target	Result	Status
Task Completion Rate	> 80%	87%	✓ Achieved
Average Time to Complete Purchase	< 5 min	3.5 min	✓ Achieved
SUS Score	> 70	78	✓ Achieved
User Satisfaction	> 4/5	4.2/5	✓ Achieved

Key Feedback:

> "Most users (8/10) found the interface intuitive. Common suggestions included adding product filters and a wishlist feature. Two users experienced confusion with the checkout flow, which was addressed by adding progress indicators."

Chapter 8: LSEPI

Legal, Social, Ethical, and Professional Issues

Show that you are a **responsible developer** who considers broader impact.

LSEPI			
LEGAL	SOCIAL	ETHICAL	PROFESSIONAL
• Data privacy • GDPR • Licenses • Terms of Service	• Social impact • Accessib-ility • Digital divide	• User consent • Data security • Bias prevention	• Clean code standards • Version control • Testing practices

8. LSEPI: Details

Legal Issues

- **Data Privacy:** GDPR/local data protection compliance
- **User consent** for data collection
- **Terms of Service** and **Privacy Policy**
- **Software Licenses:** MIT, Apache, GPL for third-party libraries

Social Issues

- **Accessibility:** WCAG guidelines, screen reader support
- **Social impact:** Who benefits? Any negative impacts?

Ethical Issues

- **Secure storage** of personal data (encrypted)
- **Transparent** data usage policies
- **No deceptive** practices (dark patterns)

Professional Issues

- **Code quality** standards followed
- **Version control** best practices
- **Thorough testing** before deployment

Chapter 9: Conclusion & Critical Reflection

Purpose

Synthesize everything. Answer: "Did you achieve what you set out to do?", "What did you learn?", and "What's next?"

Structure

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    CONCLUSION & CRITICAL REFLECTION                      │
├──────────────────────────────────────────────────────────────────────────┤
│  9.1 Summary of Achievements                                             │
│      └─→ Evaluate against objectives                                     │
│  9.2 Critical Project Evaluation                                         │
│      └─→ What went well? What didn't?                                    │
│  9.3 Personal Reflection and Key Learnings                               │
│      └─→ Your growth, skills developed                                   │
│  9.4 Recommendations for Future Development                              │
│      └─→ Feature roadmap, technical improvements                         │
│  9.5 Final Conclusion                                                    │
│      └─→ Powerful closing statement                                      │
└──────────────────────────────────────────────────────────────────────────┘
```

9.1 Summary of Achievements

Evaluate Against Objectives

Objective (from Chapter 1)	Met?	Evidence
1. Design and develop responsive web app with React/Node	✓ Yes	Fully functional application deployed at [URL]
2. Implement JWT authentication	✓ Yes	Secure login/registration system with role-based access
3. Create RESTful API for data management	✓ Yes	15+ API endpoints documented and tested
4. Integrate payment gateway	✓ Yes	Stripe integration completed with test transactions
5. Deploy and conduct UAT with 10 users	✓ Yes	Application live; UAT completed with 87% task success rate

Summary: All 5 objectives were successfully achieved. The project delivered a functional e-commerce platform meeting the defined requirements.

9.2 Critical Project Evaluation

What Went Well ✓

- **Technology choice:** React + Node.js enabled rapid development
- **Agile methodology:** Sprint reviews caught issues early
- **MongoDB:** Flexible schema allowed quick iterations

What Didn't Go Well ✗

- **Payment integration:** More complex than anticipated (2-week delay)
- **Mobile responsiveness:** Required additional optimization time
- **Initial scope:** Had to move 2 features to "Could Have"

How Challenges Were Addressed

> "The payment integration delay was mitigated by extending working hours during Sprint 4 and deferring the email notification feature (Could Have) to focus on core functionality."

9.3 Personal Reflection & Key Learnings

Be Specific About Your Growth

Technical Skills Learned

Area	What I Learned
Backend	Building RESTful APIs with Express, JWT authentication
Frontend	React hooks, state management with Context API
Database	MongoDB schema design, Mongoose ODM
DevOps	CI/CD with GitHub Actions, cloud deployment

Soft Skills Developed

- **Time management:** Gantt chart planning and sprint tracking
- **Problem-solving:** Debugging complex async issues
- **Self-learning:** Learning new libraries from documentation

What I Would Do Differently

> "I would spend more time on database design upfront to avoid schema changes mid-project. I would also start testing earlier in the process."

9.4 Recommendations for Future Development

Feature Roadmap

Priority	Feature	Description
High	Mobile App	Develop native mobile app using React Native
High	Multi-vendor	Allow multiple sellers on the platform
Medium	AI Recommendations	Product suggestions based on browsing history
Medium	Analytics Dashboard	Advanced sales and visitor analytics
Low	Multi-language	Support for additional languages

Technical Improvements

- Implement Redis caching for improved performance
- Add comprehensive unit test suite (target 90% coverage)
- Migrate to microservices architecture for scalability
- Implement WebSocket for real-time order notifications

9.5 Final Conclusion

Powerful Closing Statement

Write a short, impactful paragraph that:

1. **Restates** the core problem
2. **Summarizes** your solution
3. **Ends** with forward-looking impact

Example

> "In conclusion, this project successfully addressed the digital divide facing Vietnamese micro-businesses by developing an affordable, user-friendly e-commerce platform. The application enables local entrepreneurs with limited technical expertise to establish an online presence, manage their product catalog, and process orders efficiently.
>
> With all five objectives achieved and positive feedback from user testing, the project provides a solid foundation for future development. As Vietnam's digital economy continues to grow, this platform has the potential to empower thousands of small businesses to participate in the online marketplace, contributing to economic inclusion and local community development."

Report Checklist

Before Submission, Verify

- [ ] **Chapter 1:** Clear problem statement, aim, SMART objectives
- [ ] **Chapter 2:** Critical analysis (not just summaries), gap identified
- [ ] **Chapter 3:** User research conducted, MoSCoW requirements table
- [ ] **Chapter 4:** Methodology applied, Gantt chart included
- [ ] **Chapter 5:** Architecture diagrams, ERD, API documentation

- [ ] **Chapter 6:** Code samples, deployment evidence, GitHub commits
- [ ] **Chapter 7:** Test cases with results, UAT feedback
- [ ] **Chapter 8:** LSEPI considerations addressed
- [ ] **Chapter 9:** Objectives evaluated, honest reflection, future work
- [ ] **Overall:** Consistent formatting, citations, proofread

Common Mistakes to Avoid

Top 10 Report Pitfalls

1. ❌ Literature review is just description, not critical analysis
2. ❌ Objectives not evaluated in conclusion
3. ❌ No evidence of project management (missing Trello/GitHub screenshots)
4. ❌ Code samples without explanation
5. ❌ No user testing or feedback

6. ❌ Ignoring LSEPI section
7. ❌ Vague reflection ("I learned a lot")
8. ❌ Missing diagrams (ERD, architecture)
9. ❌ Inconsistent formatting and citations
10. ❌ Submitting without proofreading

Final Tips for Success

Keys to a Strong Report

✓ **Be Critical** - Analyze, don't just describe
✓ **Be Honest** - Acknowledge challenges and limitations
✓ **Be Specific** - Use concrete examples and evidence
✓ **Be Professional** - Consistent formatting, proper citations
✓ **Be Reflective** - Show genuine learning and growth

Resources

- Consult your supervisor regularly
- Review successful past reports (if available)
- Use Grammarly or similar for proofreading
- Start writing early - good reports take time!

Questions?

Good luck with your projects!